

Project 1 报告：曲线和曲面造型技术

课程：计算机图形学 COMP130018.01 姓名：庞嘉、朱程炀 学号：23307130277、
23300240014 日期：2026年4月

一、任务概述

本次 Project 1 的目标是实现样条曲线（Bézier 曲线、B 样条曲线）及基于曲线的扫掠曲面（旋转曲面、广义圆柱体），并解决闭合曲线处坐标系不连续的问题。主要工作分为三个部分：

- 任务 1：在 `curve.cpp` 中实现 `evalBezier` 与 `evalBspline` 函数，生成曲线点列及局部坐标系 (T, N, B)。
- 任务 2：在 `surf.cpp` 中实现 `makeSurfRev` 与 `makeGenCyl` 函数，生成带法向量的三角网格曲面。
- 拓展：实现闭合曲线坐标系修正，消除 `weirder.swp` 的接缝不连续现象。

二、任务 1：曲线的绘制

2.1 分段三次 Bézier 曲线

2.1.1 曲线求值

给定 $3n + 1$ 个控制点 $P_0, P_1, \dots, P_{3n}$ ，每四个点 $(P_{3i}, P_{3i+1}, P_{3i+2}, P_{3i+3})$ 定义一段三次 Bézier 曲线。利用 Bernstein 基函数，参数 $t \in [0, 1]$ 处的位置为：

$$\mathbf{V}(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3$$

切向量（一阶导数）为：

$$\mathbf{V}'(t) = 3[(1-t)^2(P_1 - P_0) + 2(1-t)t(P_2 - P_1) + t^2(P_3 - P_2)]$$

单位切向量 $\mathbf{T} = \mathbf{V}'(t) / \|\mathbf{V}'(t)\|$ 。

也可等价地写成矩阵形式：

$$P(t) = [P_0, P_1, P_2, P_3] \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

2.1.2 局部坐标系 (T, N, B) 的构建

在三维曲线中，不能用固定的次法线初始化法线，否则会出现奇异。采用平行输运 (Parallel Transport) 方法，通过递归更新避免法线突变：

1. 初始化：在第一个点处，选取与 $\mathbf{T}_0$ 不平行的参考向量（优先选 $(0, 0, 1)$ ，若近似平行则改用 $(0, 1, 0)$ ）。用 **Gram-Schmidt** 正交化得到初始次法线：

$$\mathbf{B}_0 = \frac{\mathbf{a} - (\mathbf{a} \cdot \mathbf{T}_0)\mathbf{T}_0}{\|\mathbf{a} - (\mathbf{a} \cdot \mathbf{T}_0)\mathbf{T}_0\|}$$

这确保了对于 **XY** 平面上的曲线， $\mathbf{B}_0 = (0, 0, 1)$ ，与 **evalCircle** 的坐标系约定一致（**N** 在平面内，**B** 朝 $+z$ ）。

2. 递归更新（每一步）：

$$\mathbf{N}_i = (\mathbf{B}_{i-1} \times \mathbf{T}_i).normalized()$$

$$\mathbf{B}_i = (\mathbf{T}_i \times \mathbf{N}_i).normalized()$$

该方法允许 $\mathbf{T}_i$ 处的法向量尽量接近 $\mathbf{T}_{i-1}$ 处的法向量，同时保持与切向量正交。

2.2 B 样条曲线

B 样条曲线不直接插值控制点，每段由四个连续控制点 $P_i, P_{i+1}, P_{i+2}, P_{i+3}$ 定义：

$$P(s) = [P_i, P_{i+1}, P_{i+2}, P_{i+3}] \frac{1}{6} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 4 & 0 & -6 & 3 \\ 1 & 3 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ s \\ s^2 \\ s^3 \end{bmatrix}, \quad s \in [0, 1]$$

实现策略：将每段 **B** 样条控制点转换为等价的 **Bézier** 控制点，再复用 **evalBezier**：

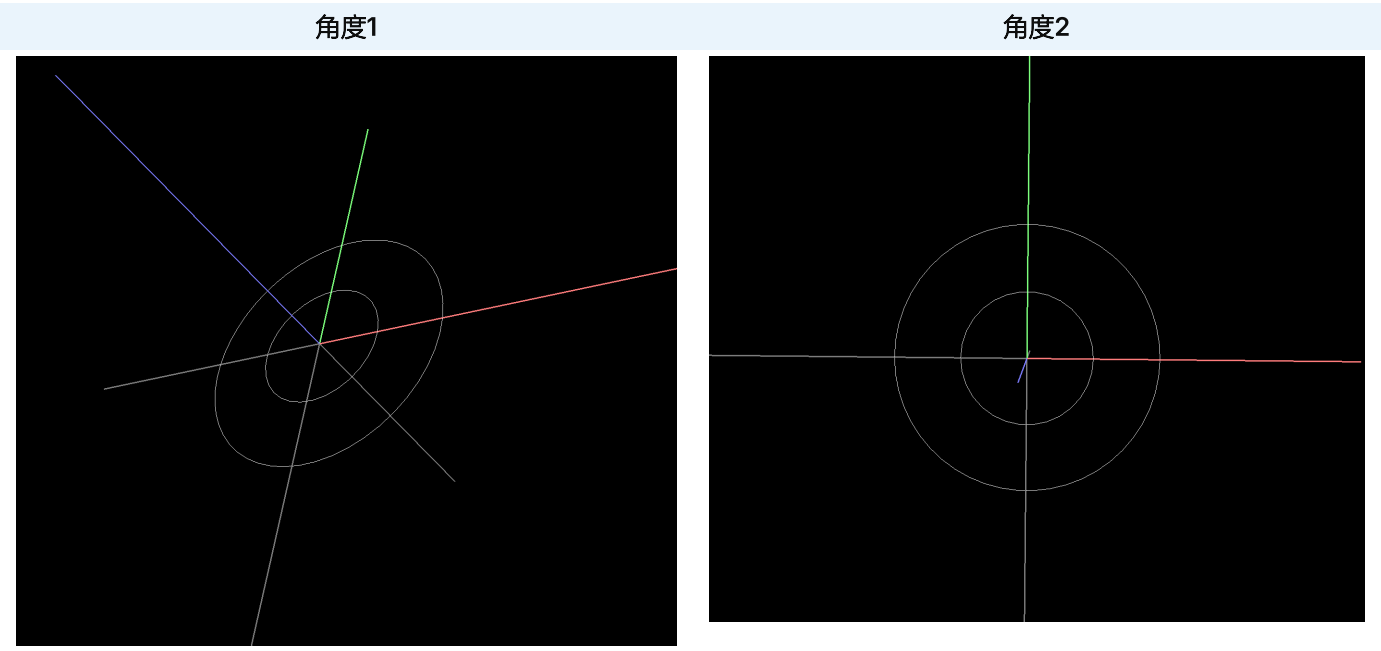
$$\mathbf{c}_0 = \frac{b_0 + 4b_1 + b_2}{6}, \quad \mathbf{c}_1 = \frac{2b_1 + b_2}{3}, \quad \mathbf{c}_2 = \frac{b_1 + 2b_2}{3}, \quad \mathbf{c}_3 = \frac{b_1 + 4b_2 + b_3}{6}$$

相邻段共享端点（ $\mathbf{c}_3$ 即下一段的 $\mathbf{c}_0$ ），保证 C^2 连续。坐标系通过段间传递 **prevB** 保持连续性。

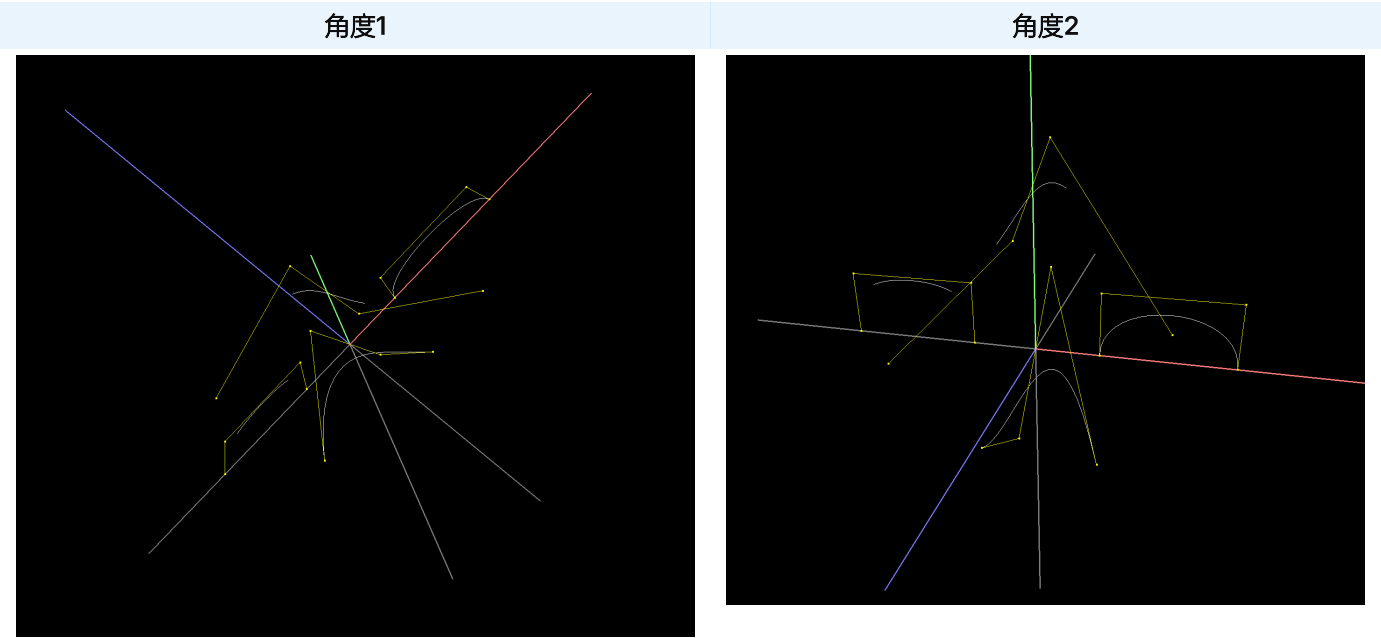
2.3 曲线实验结果

说明：每个案例左图为本实现结果，右图为参考答案
(**sample_solution/athena/a1**)。

circles.swp



core.swp



三、任务 2：曲面的绘制

3.1 旋转曲面 (Surface of Revolution)

旋转曲面通过将 **XY** 平面上的轮廓曲线绕 **Y** 轴旋转 2π 生成。旋转矩阵为：

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

顶点变换：轮廓点 $(p_x, p_y, 0)$ 绕 **Y** 轴旋转 θ 后：

$$\mathbf{V}' = (p_x \cos \theta, p_y, -p_x \sin \theta)$$

法向量变换：法向量通过变换矩阵的逆转置变换。对于纯旋转矩阵（正交矩阵， $M^{-T} = M$ ），法向量旋转方式与顶点相同。但需要反转曲线法线方向以使曲面法线朝外（轮廓曲线法线指向旋转轴，曲面法线应远离旋转轴）：

$$\mathbf{N}'_{\text{surf}} = \text{normalize}(-n_x \cos \theta, -n_y, n_x \sin \theta)$$

三角面生成：相邻旋转环之间按 **ACB**、**BCD** 顺序（逆时针，保证法线朝外）生成三角形。设 j 环、轮廓点 i 的顶点索引为 $A = j \cdot n + i$ ，则：

- 面1: (A, C, B) 其中 $C = (j+1) \cdot n + i$, $B = j \cdot n + i + 1$
- 面2: (B, C, D) 其中 $D = (j+1) \cdot n + i + 1$

3.2 广义圆柱体 (Generalized Cylinder)

广义圆柱体通过将轮廓曲线 (**Profile**) 按扫掠曲线 (**Sweep**) 的局部坐标系 ($\mathbf{N}_j, \mathbf{B}_j, \mathbf{T}_j, \mathbf{V}_j$) 变换得到。

变换矩阵为：

$$M_j = \begin{bmatrix} \mathbf{N}_j & \mathbf{B}_j & \mathbf{T}_j & \mathbf{V}_j \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

轮廓点 $(p_x, p_y, 0)$ 变换到世界坐标：

$$\mathbf{V}'_{\text{world}} = p_x \mathbf{N}_j + p_y \mathbf{B}_j + \mathbf{V}_j$$

法向量（轮廓法线 $(n_x, n_y, 0)$ ）变换后取反以朝外：

$$\mathbf{N}'_{\text{world}} = \text{normalize}(-(n_x \mathbf{N}_j + n_y \mathbf{B}_j))$$

由于旋转矩阵 $[\mathbf{N}_j, \mathbf{B}_j, \mathbf{T}_j]$ 是正交矩阵，其逆转置就是它本身，故法向量变换方式与位置相同（忽略平移部分）。

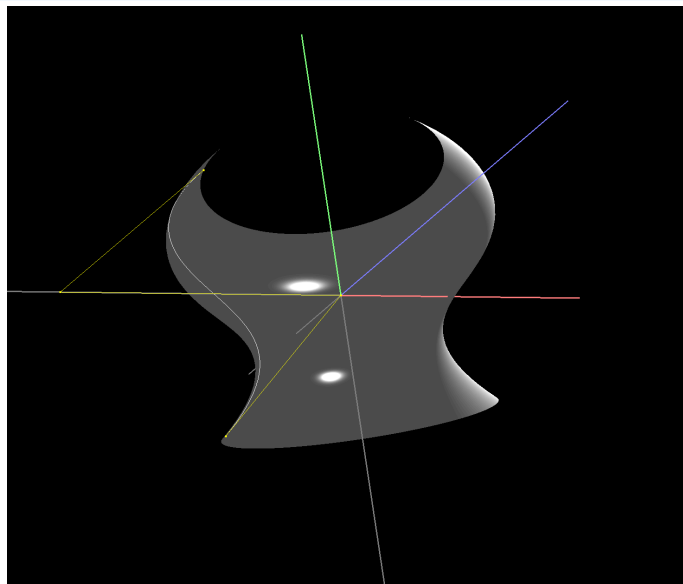
三角面生成方式与旋转曲面相同，复用了同一辅助函数 **addFaces**。

3.3 曲面实验结果

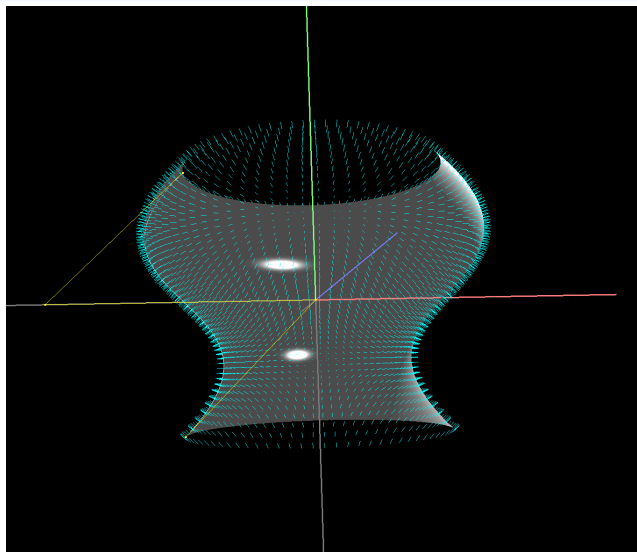
说明：每个案例左图为本实现结果（曲面着色模式），右图为参考答案。可按 **S** 键切换曲面/线框/法线显示模式。

norm.swp

角度1

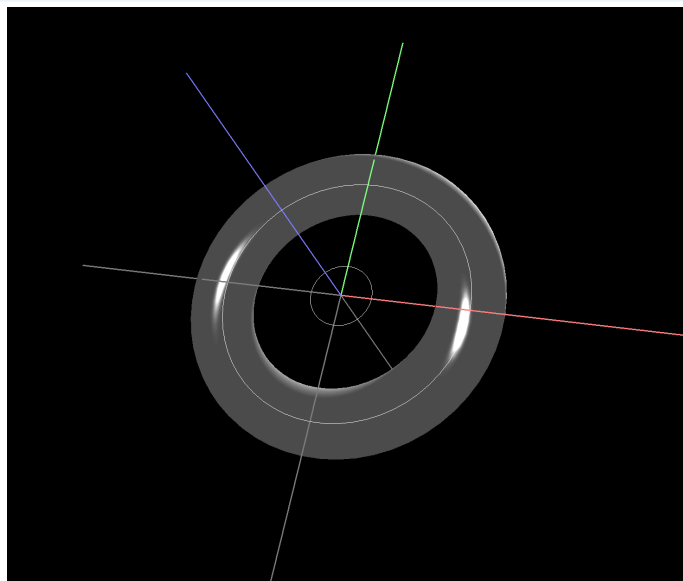


角度2

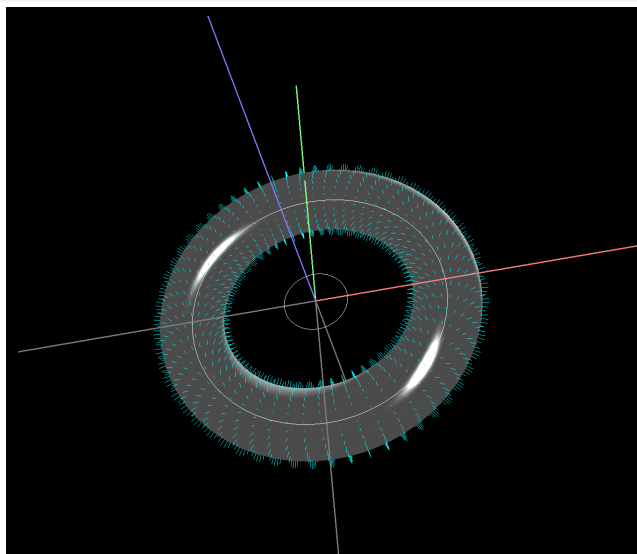


tor.swp

角度1

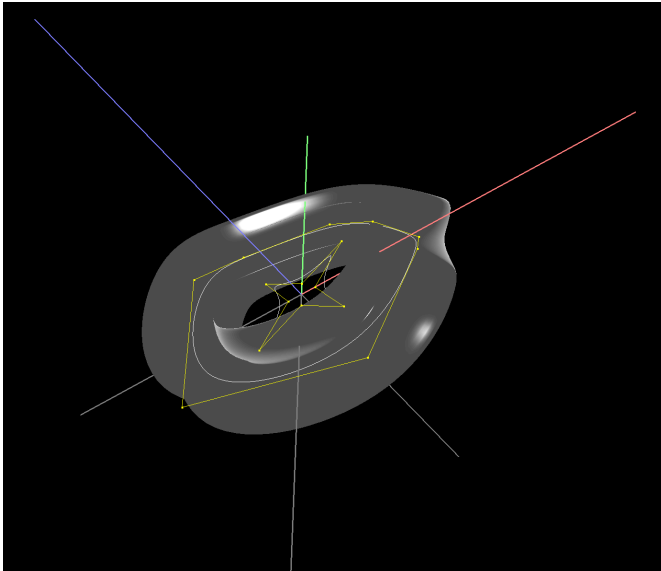


角度2

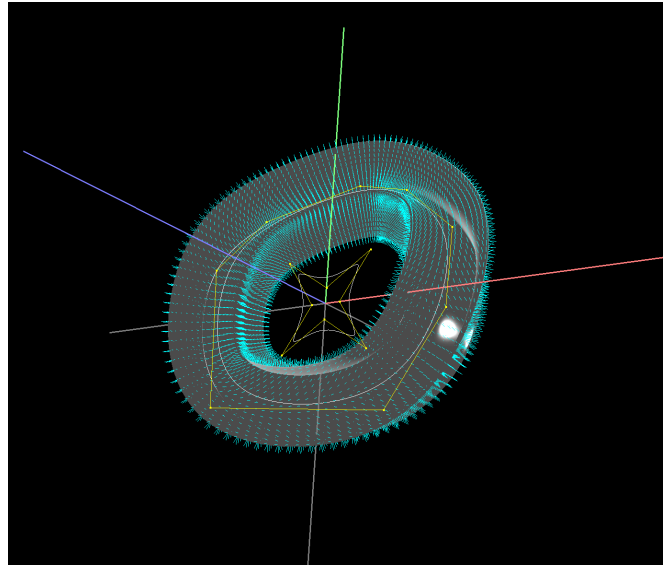


weird.swp

角度1

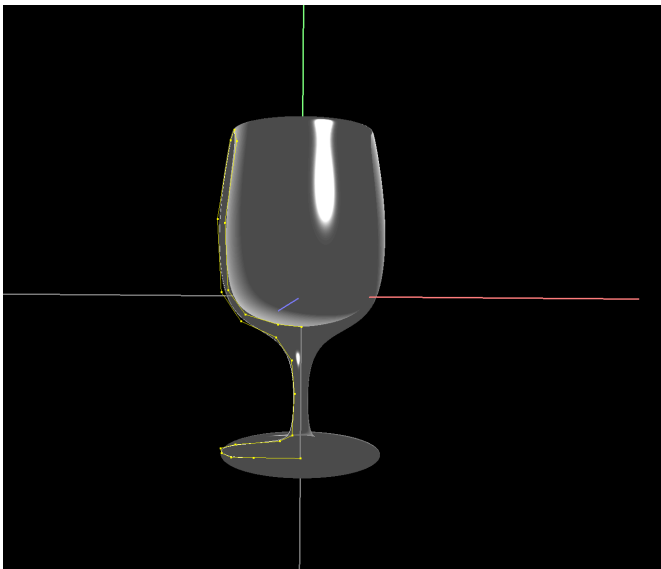


角度2

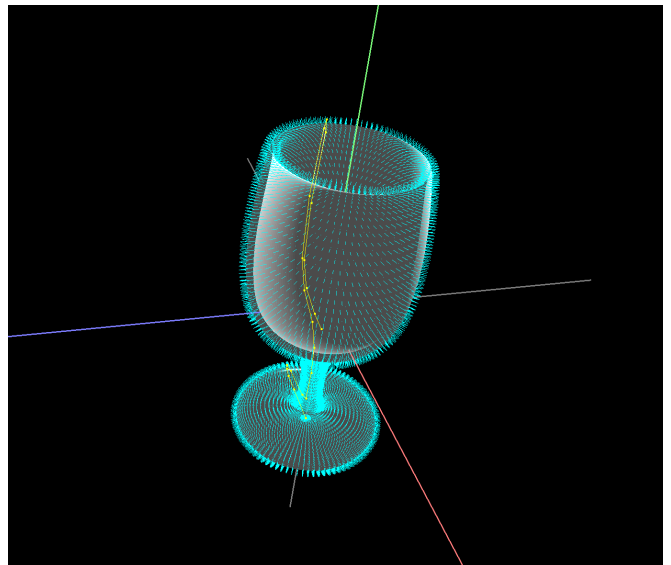


wineglass.swp

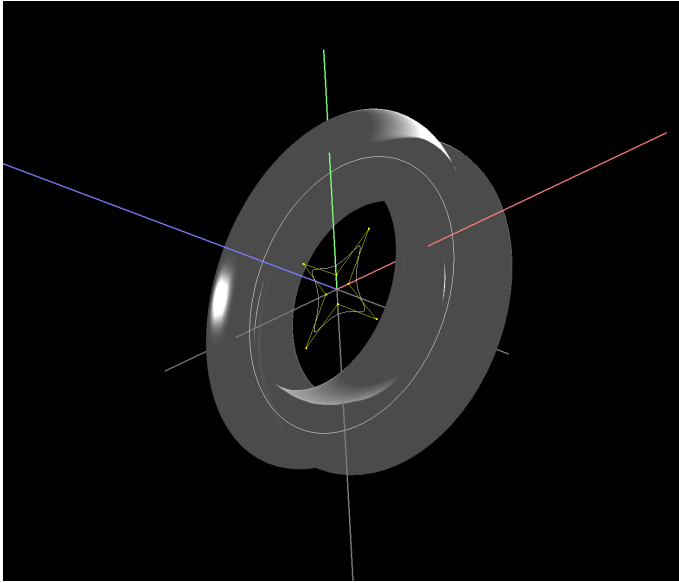
本实现



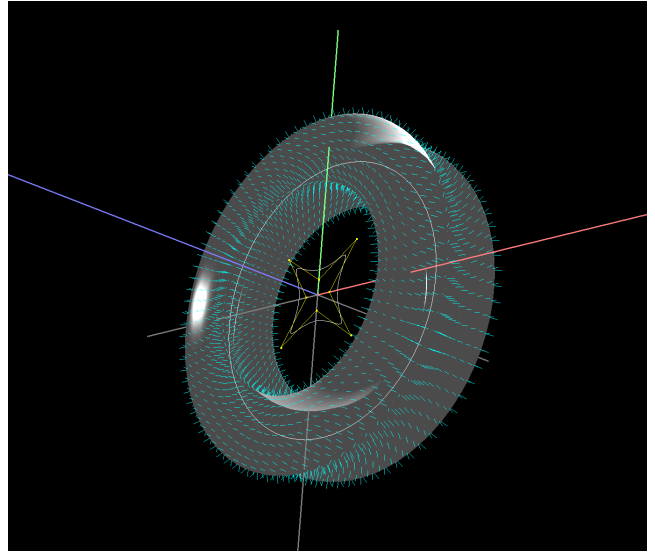
参考答案



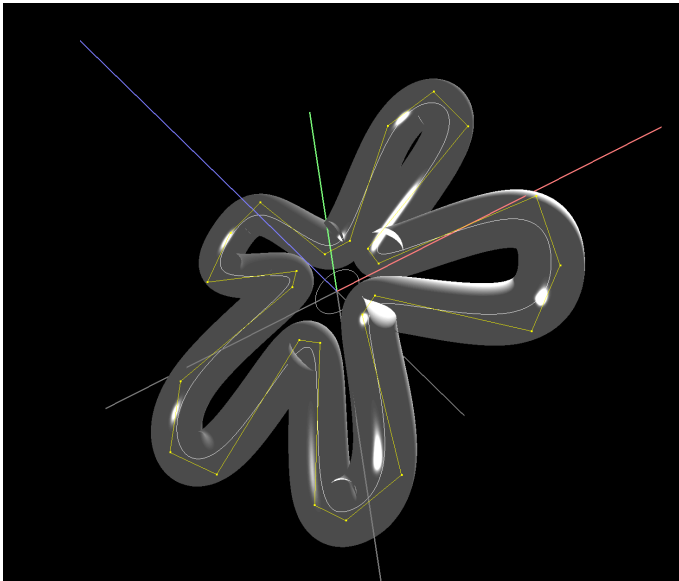
角度1



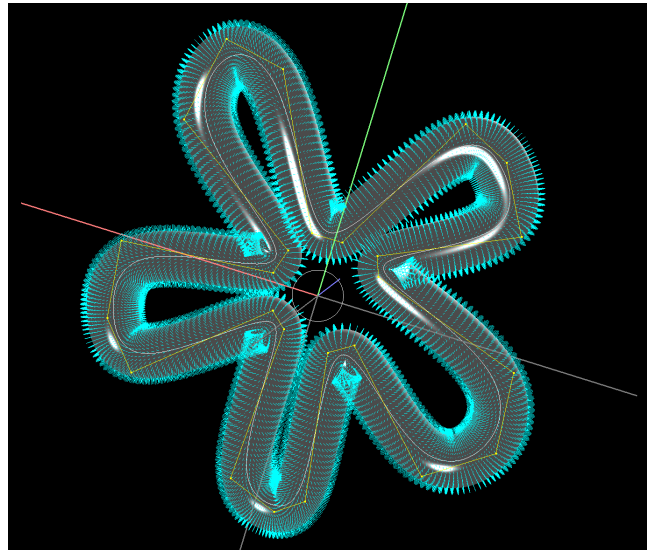
角度2

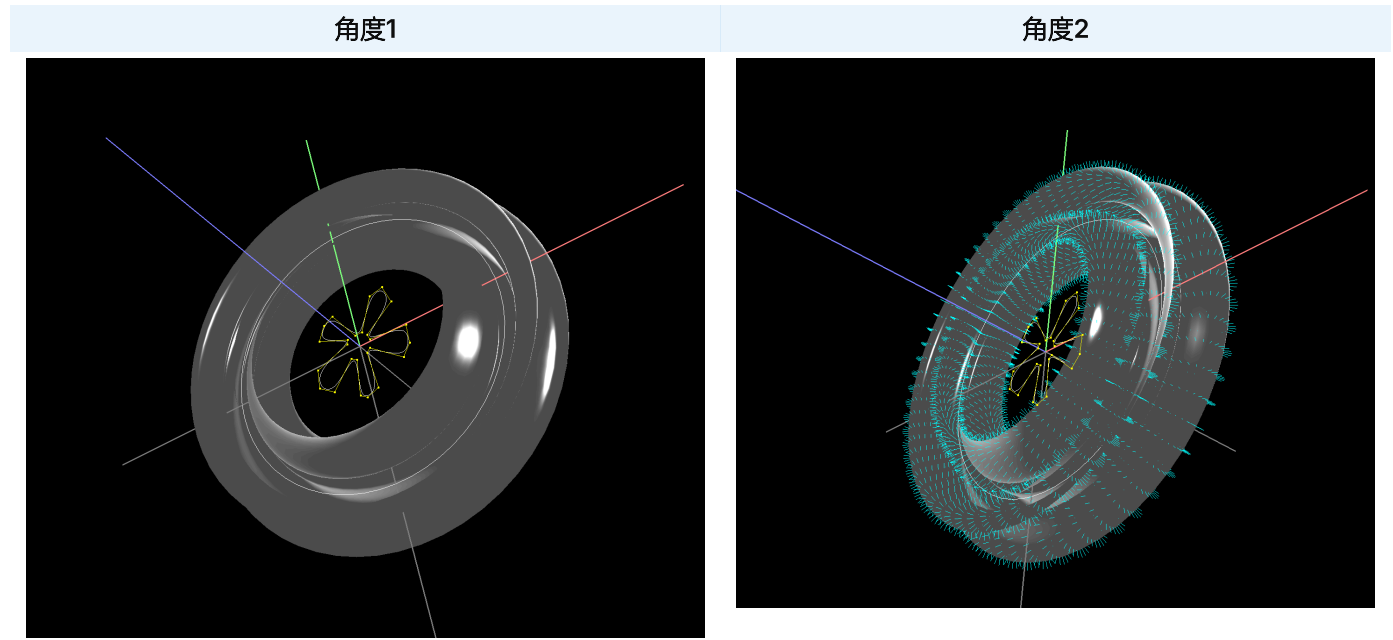


角度1



角度2





四、拓展：曲面闭合问题的解决

4.1 问题分析

对闭合扫掠曲线（起始与终止位置、切向量均相同），平行输运方法会沿曲线积累一个扭转角 α ：

$$\alpha = \angle(\mathbf{N}_0, \mathbf{N}_{n-1} \text{ 绕 } \mathbf{T}_0)$$

由于该扭转，首尾环的坐标系存在 α 的旋转偏差，导致 **GenCyl** 曲面在接缝处出现不连续（撕裂）。

4.2 解决方案

检测闭合条件：首尾位置距离 $< \varepsilon$ 且 $\mathbf{T}_0 \cdot \mathbf{T}_{n-1} \approx 1$ 。

计算扭转角（有符号，右手定则，绕 $\mathbf{T}$ 为正方向）：

$$\begin{aligned} \cos \alpha &= \mathbf{N}_0 \cdot \mathbf{N}_{n-1}, & \sin \alpha &= (\mathbf{N}_0 \times \mathbf{N}_{n-1}) \cdot \mathbf{T}_0 \\ \alpha &= \text{atan2}(\sin \alpha, \cos \alpha) \end{aligned}$$

分布修正旋转：为消除接缝，在第 i 个点处施加累计修正旋转角：

$$\theta_i = -\alpha \cdot \frac{i}{n-1}$$

- $i = 0$: $\theta_0 = 0$, 起点不变

- $i = n - 1$: $\theta_{n-1} = -\alpha$, 精确抵消积累扭转, 使末尾帧对齐起始帧

罗德里格公式 (绕 $\mathbf{T}_i$ 旋转 θ_i , 由于 $\mathbf{N} \perp \mathbf{T}$ 、 $\mathbf{B} \perp \mathbf{T}$, 公式化简为) :

$$\mathbf{N}'_i = \cos \theta_i \cdot \mathbf{N}_i + \sin \theta_i \cdot \mathbf{B}_i$$

$$\mathbf{B}'_i = -\sin \theta_i \cdot \mathbf{N}_i + \cos \theta_i \cdot \mathbf{B}_i$$

其中 $\mathbf{B}_i = \mathbf{T}_i \times \mathbf{N}_i$, 即 $\mathbf{T} \times \mathbf{N} = \mathbf{B}$ (右手坐标系)。

正确性验证: 设末尾点已被平行输运扭转 α , 即 $\mathbf{N}_{n-1} = \cos \alpha \mathbf{N}_0 + \sin \alpha \mathbf{B}_0$, 施加修正 $\theta = -\alpha$:

$$\mathbf{N}'_{n-1} = \cos(-\alpha)\mathbf{N}_{n-1} + \sin(-\alpha)\mathbf{B}_{n-1} = (\cos^2 \alpha + \sin^2 \alpha)\mathbf{N}_0 = \mathbf{N}_0$$

对非闭合曲线无影响: 若位置或切向量不满足闭合条件, 函数直接返回, 不对任何帧进行修改。

4.3 关键代码 (curve.cpp)

C++

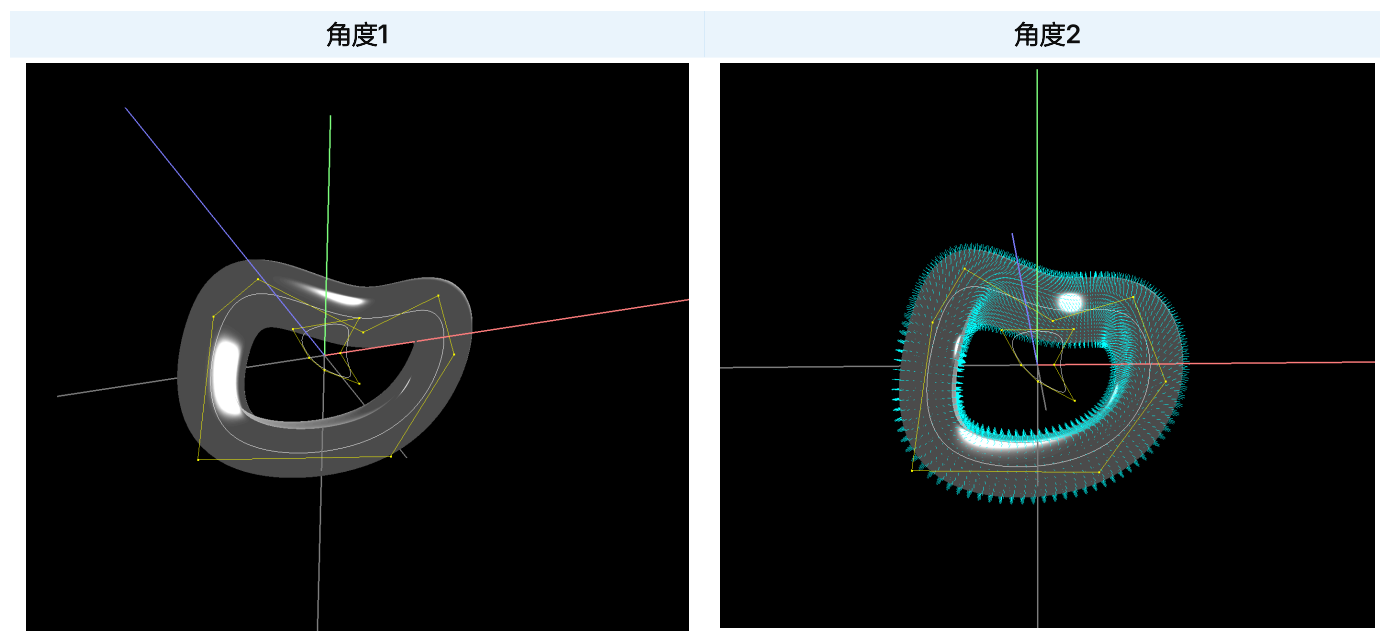
```
void fixClosedCurve(Curve& curve) {
    // 1. 闭合检测
    const float eps = 1e-4f;
    if ((first.V - last.V).absSquared() > eps) return;
    if (fabs(Vector3f::dot(first.T, last.T) - 1.0f) > eps) return;

    // 2. 计算有符号扭转角 alpha (从 first.N 到 last.N 绕 first.T)
    float cosA = Vector3f::dot(first.N, last.N);
    float sinA = Vector3f::dot(Vector3f::cross(first.N, last.N), first.T);
    float alpha = atan2(sinA, cosA);

    // 3. 对每个点施加线性插值的反向修正旋转 -alpha * i / (n-1)
    for (int i = 0; i < total; i++) {
        float theta = -alpha * float(i) / float(total - 1);
        // 罗德里格公式: 绕 T 旋转 theta
        curve[i].N = cosT * Ni + sinT * Bi;
        curve[i].B = -sinT * Ni + cosT * Bi;
    }
}
```

4.4 实验结果

weirder.swp



五、总结

主要难点与解决方式：

- 平面曲线帧约定：原始代码用 `cross(arb, T)` 初始化 `prevB`，导致 XY 平面曲线的 `N[2]≠0`，触发 `checkFlat` 检查失败。改用 Gram-Schmidt 正交化后，平面曲线的 `B=(0,0,1)`，`N` 在平面内，与 `evalCircle` 约定一致。
- 闭合修正符号：扭转角 α 是从 `first.N` 到 `last.N` 的角度（积累量），修正需用 $-\alpha$ （反向）；原代码误用 $+\alpha$ 使末尾点扭转加倍，改为负号后接缝消除。

Tip

还加入了 `run_tests.sh` 脚本，在 `/starter1` 目录下，使用 `bash tests/run_tests.sh` 即可运行脚本，与标准答案进行对比。